

Complete reference on `var` , `let` , and `const` — covering scope, hoisting, mutability, the event loop, and more.

Quick Comparison Table

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Scope	Function	Block	Block
Hoisting	Yes (undefined)	TDZ	TDZ
Attached to <code>window</code>	Yes	No	No

1. Redeclaration

Redeclaration means declaring a variable with the same name twice in the same scope.

```
// var - allowed (dangerous, can cause silent bugs)
var x = 1;
var x = 2; // no error
console.log(x); // 2

// let - NOT allowed
let y = 1;
let y = 2; // SyntaxError: Identifier 'y' has already been declared

// const - NOT allowed
const z = 1;
const z = 2; // SyntaxError
```

Why does this matter? With `var`, you can accidentally overwrite a variable you didn't mean to. `let` and `const` protect you from this mistake at compile time.

2. Reassignment

Reassignment means changing the value of an already-declared variable.

```
// var - fully reassignable
var x = 1;
x = 2;
x = true;
x = "hello";
console.log(x); // "hello"

// let - fully reassignable
let y = 1;
y = true;
y = "hello";
console.log(y); // "hello"

// const - NOT reassignable
const z = 1;
z = 4; // TypeError: Assignment to constant variable
```

Rule of thumb: Use `const` by default. Switch to `let` only when you know the value must change (counters, loop variables, toggling state). Avoid `var`.

3. Immutable vs Mutable

These two words describe whether a value can be changed after it's created.

Strings are Immutable

```
var fname = "mona";
fname[0] = "M";
```

```
console.log(fname); // "mona" - unchanged! strings are immutable in JS
```

Strings cannot be mutated in place. To "change" a string, you must create a new one:

```
var fname = "mona";  
fname = fname.slice(0, 1).toUpperCase() + fname.slice(1);  
console.log(fname); // "Mona"
```

Numbers are Immutable (but Variables are Reassignable)

```
var age = 30;  
age = 31; // this replaces the value, not mutates it
```

Objects and Arrays are Mutable

```
var user = { name: "mona", age: 20 };  
user.name = "abdullah"; // modifies the object in place  
console.log(user); // { name: "abdullah", age: 20 }
```

Key insight: `const` prevents reassignment of the *variable*, but it does NOT prevent mutation of the *object or array* the variable points to.

```
const user = { name: "mona", age: 20 };  
user = {}; // TypeError - can't reassign the binding  
user.name = "abdullah"; // ALLOWED - mutating the object is fine  
user.email = "abdullah@gmail.com"; // ALLOWED  
console.log(user); // { name: "abdullah", age: 20, email:  
"abdullah@gmail.com" }
```

```
const arr = [1, 2, 3, 4, 5];  
arr = []; // TypeError  
arr[0] = 100; // ALLOWED  
arr.push(70); // ALLOWED  
console.log(arr); // [100, 2, 3, 4, 5, 70]
```

4. Object.freeze()

`Object.freeze()` prevents any modifications to an object or array — it makes it **truly immutable**.

```
var arr = [1, 2, 3, 4, 5];
Object.freeze(arr);
arr[0] = 100; // silently fails (or throws in strict mode)
arr.push(70); // TypeError in strict mode
console.log(arr); // [1, 2, 3, 4, 5]
```

```
var user = { name: "mona", age: 20 };
Object.freeze(user);
user.name = "abdullah"; // silently fails
user.email = "abdullah@gmail.com"; // silently fails
console.log(Object.isFrozen(user)); // true
```

freeze() vs const

	<code>const</code>	<code>Object.freeze()</code>
Prevents reassignment	Yes	No
Prevents mutation	No	Yes
Works on primitives	Yes	N/A
Works on objects/arrays	Only binding	Deep protection

Note: `Object.freeze()` is **shallow** — nested objects inside a frozen object are NOT frozen.

```
const obj = { a: 1, nested: { b: 2 } };
Object.freeze(obj);
obj.nested.b = 99; // ALLOWED - nested is not frozen
```

`const` must always have an initial value. `const x;` is a `SyntaxError`.

5. Scope

Scope determines where in your code a variable is accessible.

Global Scope

Variables declared outside any function or block are globally accessible.

```
var x = 1;
let y = 2;
const z = 3;

function scope() {
  if (true) {
    console.log(x, y, z); // 1, 2, 3 - all accessible
  }
}
scope();
```

Function Scope

`var`, `let`, and `const` declared inside a function are only accessible within that function.

```
function scopeExample() {
  var x = 1;
  let y = 2;
  const z = 3;
  console.log(x, y, z); // 1, 2, 3

  if (true) {
    console.log(x, y, z); // 1, 2, 3 - still inside the function
  }
}
scopeExample();
```

Block Scope

A block is anything wrapped in `{}` — `if`, `for`, `while`, etc.

```
if (true) {
  var x = 1;    // function-scoped (leaks out!)
  let y = 2;    // block-scoped
  const z = 3;  // block-scoped
}

console.log(x); // 1 - accessible (var leaked)
console.log(y); // ReferenceError - not accessible
console.log(z); // ReferenceError - not accessible
```

`var` ignores block boundaries. This is one of the main reasons it's considered dangerous.

Complex Scope Example

```
let x = 1;

function scopeExample() {
  const y = 5;

  if (true) {
    var z = 3;  // hoisted to function scope
    let a = 4;  // block-scoped
  }

  console.log(z); // 3 - var leaked out of the if block
  console.log(a); // ReferenceError - let is block-scoped
}

scopeExample();
```

6. Hoisting

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

var is Hoisted (with `undefined`)

```
console.log(x); // undefined (not an error!)
var x = 1;
```

JavaScript sees this as:

```
var x;           // declaration hoisted to top
console.log(x); // undefined
x = 1;           // assignment stays in place
```

let and const — Temporal Dead Zone (TDZ)

`let` and `const` are also hoisted, but they are placed in the **Temporal Dead Zone (TDZ)** — a region where the variable exists but cannot be accessed.

```
function test() {
  console.log(x); // ReferenceError: Cannot access 'x' before
  initialization
  let x = 5;
}
test();
```

The TDZ exists from the start of the scope until the line where the variable is declared. Accessing a variable in the TDZ throws a `ReferenceError`.

Why TDZ is Better

`var`'s behavior of silently returning `undefined` can hide bugs. `let` / `const` throw immediately, so you discover mistakes faster.

7. Attached to Global Object (window)

In browsers, the global object is `window`. Variables declared with `var` at the global level become properties of `window`.

```
var x = 1;
console.log(x, window.x); // 1 1 - same thing
```

```
let y = 1;
const z = 2;
console.log(window.y, window.z); // undefined undefined
```

This means `var` can accidentally pollute or overwrite global browser APIs. For example, if you `var name = "mona"` at the top level, you just overwrote `window.name`.

8. The Event Loop & `setTimeout` with `var` vs `let`

This is one of the most famous JavaScript interview questions.

The Problem with `var`

```
var i;
for (i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
// Output after 1 second: 3, 3, 3
```

Why? `var` is function-scoped, so there is only ONE `i` shared across all iterations. By the time `setTimeout` fires (after 1 second), the loop has finished and `i` is already `3`.

The Fix with `let`

```
for (let i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
// Output after 1 second: 0, 1, 2
```


Why? `let` is block-scoped. Each iteration of the loop creates a **new, separate** `i` that is captured by the `setTimeout` callback. Each closure closes over its own `i`.

Understanding the Event Loop

```
console.log("First");
setTimeout(() => {
  console.log("Second");
}, 1000);
console.log("Third");
```

Output:

```
First
Third
Second  ← appears after ~1 second
```

How the Event Loop works:

1. **Call Stack** — JavaScript executes synchronous code line by line.
2. **Web APIs** — `setTimeout` is handed off to the browser's timer API (not JS itself).
3. **Callback Queue** — After 1 second, the callback is placed in the queue.
4. **Event Loop** — Continuously checks: "Is the call stack empty?" If yes, it moves the next callback from the queue to the stack.

JavaScript is **single-threaded** but handles asynchronous operations via the event loop, allowing non-blocking behavior.

9. Best Practices Summary

```
const → Use by default for everything
let   → Use only when reassignment is needed (loops, counters,
toggling)
var    → Avoid completely in modern JavaScript
```

- Prefer `const` — it signals intent: "this value won't change"

- Use `let` for loop counters, accumulators, and values that need to update
 - Never use `var` — it has too many quirks (hoisting, function scope, window pollution)
 - Use `Object.freeze()` when you need truly immutable objects
 - Always initialize `const` variables — they must have a value at declaration
-

Abdullah Ali

Contact : +201012613453